

Deploy AI Agent to Google Cloud Run

This guide provides step-by-step instructions on how to deploy the Pinion AI chat application to Google Cloud Run, including configuration of required environment variables using a `.env` file, or `env.yaml`.

Prerequisites

Before you begin, ensure you have the following installed and configured:

1. **Google Cloud SDK (gcloud CLI):** Google Cloud CLI is installed and configured.
2. **Docker:** Docker is installed.
3. A Google Cloud Project. If you don't have one, create one in the Google Cloud Console.

Log in to your Google Cloud account and set your project.

```
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```

Replace `YOUR_PROJECT_ID` with your actual Google Cloud project ID.

Step 1: Export out a few core variables

```
# Set basic environment variables for your project and desired service
account name
export PROJECT_ID=$(gcloud config get-value project)
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
```

Step 2: Create a Dedicated Service Account and Grant Necessary IAM Roles

- NOTE: only do step 2 once

First, create a new, dedicated service account for your Cloud Run service. This ensures the service has only the permissions it needs, following the principle of least privilege.

```
# Create the service account
gcloud iam service-accounts create "${SERVICE_ACCOUNT_NAME}" \
  --display-name="Cloud Run PinionAI Client Service Account" \
  --project="${PROJECT_ID}"
```

Next, grant the required IAM roles to the new service account. These permissions allow the service to interact with Vertex AI, Cloud Storage, and use the enabled APIs.

```
# Get the full email address of the service account
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"

# To execute Gemini prompts and use RAG Stores
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
  --role="roles/aiplatform.user"

# Grant the Storage Object Admin role for uploading files to GCS
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
  --role="roles/storage.objectAdmin"

# Grant the Service Usage Consumer role to allow using the APIs
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
  --role="roles/serviceusage.serviceUsageConsumer"

# Grant Cloud Run invoker role
gcloud projects add-iam-policy-binding "${PROJECT_ID}" \
  --member="serviceAccount:${SERVICE_ACCOUNT_EMAIL}" \
  --role="roles/run.invoker"
```

Deployment Steps

1. Set Up Your Environment

Define some environment variables to make the following commands easier to manage. Choose a region that supports Cloud Run.

```
export PROJECT_ID=$(gcloud config get-value project)
export REGION="us-central1" # Or your preferred region
export REPOSITORY="pinionai-chat-repo"
export IMAGE_NAME="pinionai-chat"
export SERVICE_ACCOUNT_NAME="pinionai-client-runner"
export
SERVICE_ACCOUNT_EMAIL="${SERVICE_ACCOUNT_NAME}@${PROJECT_ID}.iam.gserviceac
count.com"
```

2. Enable Required APIs (only need to do this once)

You need to enable the Artifact Registry API (to store your Docker image) and the Cloud Run API (to run your service).

```
gcloud services enable artifactregistry.googleapis.com run.googleapis.com
```

3. Create an Artifact Registry Repository (only need to do this once)

Create a Docker repository in Artifact Registry to host your container image.

```
gcloud artifacts repositories create ${REPOSITORY} \
  --repository-format=docker \
  --location=${REGION} \
  --description="Docker repository for the PINION AI Chat application"
```

4. Configure Docker Authentication

Configure the Docker command-line tool to authenticate with Artifact Registry.

```
gcloud auth configure-docker ${REGION}-docker.pkg.dev
```

5. Build and Push the Docker Image

Build the Docker image from the **Dockerfile** in the project root, tag it, and push it to your Artifact Registry repository.

```
# Define the full image URI
export IMAGE_URI="${REGION}-
docker.pkg.dev/${PROJECT_ID}/${REPOSITORY}/${IMAGE_NAME}:latest"

# As a deploy (single command to build and push)
gcloud builds submit --tag ${IMAGE_URI}

# --- OR IF you want to do these 2 steps separately ----
# Build the Docker image from the project root
docker build -t ${IMAGE_URI} .

# Push the image to Artifact Registry
docker push ${IMAGE_URI}
```

6. Deploy to Google Cloud Run with Environment Variables

Deploy your container image to Cloud Run and pass the environment variables from your **.env** file. You can specify each variable using the **--set-env-vars** flag, Or use the **--env-vars-file** flag with a YAML file.

Use the **--service-account** flag to specify the identity your service will run as.

Option 1: Using `--set-env-vars`

Extract the variables from your `.env` file and pass them as a comma-separated list:

```
gcloud run deploy ${IMAGE_NAME} \
  --image=${IMAGE_URI} \
  --port=8080 \
  --service-account=${SERVICE_ACCOUNT_EMAIL} \
  --project=${PROJECT_ID} \
  --region=${REGION} \
  --platform=managed \
  --allow-unauthenticated \
  --min-instances 0 \
  --cpu-boost \
  --set-env-vars
client_id=YOUR_CLIENT_ID_HERE,client_secret=YOUR_CLIENT_SECRET_HERE,etc...
```

- In `--set-env-vars`, to run a specific agent, environment variables needed are:

```
client_id = '<YOUR_CLIENT_ID_HERE>'
client_secret = '<YOUR_CLIENT_SECRET_HERE>'
agent_id = '<YOUR_AGENT_ID_HERE>'
host_url = 'https://microservice-72loomfx5q-uc.a.run.app' #
'<PINIONAI_API_HOST_URL_HERE>'
```

OR USE ENV YAML

- For a generic app that allows any agent to be loaded via `.AIA` file (`ai agent`) file, you do not need `client_id`, `client_secret` or a specific `agent_id`.

```
host_url = 'https://microservice-72loomfx5q-uc.a.run.app' #
'<PINIONAI_API_HOST_URL_HERE>'
```

Option 2: Using an Environment Variables YAML File

Convert your `.env` file to a YAML file (e.g., `env.yaml`):

```
client_id: <YOUR_CLIENT_ID_HERE>
client_secret: <YOUR_CLIENT_SECRET_HERE>
agent_id: <YOUR_AGENT_ID_HERE>
host_url: https://microservice-72loomfx5q-uc.a.run.app
```

Then deploy with:

```
gcloud run deploy ${IMAGE_NAME} \  
  --image=${IMAGE_URI} \  
  --port=8080 \  
  --service-account=${SERVICE_ACCOUNT_EMAIL} \  
  --project=${PROJECT_ID} \  
  --region=${REGION} \  
  --platform=managed \  
  --allow-unauthenticated \  
  --min-instances 0 \  
  --cpu-boost \  
  --env-vars-file deploy/prod-agent/env.yaml
```

OR

- For a generic app that allows any agent to be loaded via .AIA file (**ai agent**) file, you do not need `client_id`, `client_secret` or a specific `agent_id`. Use a different `env.yaml` file, for example the one found here.

```
--env-vars-file deploy/prod-aia-file/env.yaml
```

During the deployment process, **gcloud** will prompt you to confirm the settings. Once you confirm, it will deploy the service and provide you with a **Service URL**.

Important Note: Vertex AI Service Agent Permissions

For PinionAI Studio RAG file import to succeed, the **Vertex AI Service Agent** (a Google-managed service account) must have permission to read files from your Google Cloud Storage bucket.

This permission is usually granted automatically when you enable the Vertex AI API. However, if you encounter **Permission Denied** errors during ingestion, ensure the Vertex AI Service Agent has the **Storage Object Viewer** (`roles/storage.objectViewer`) role on your project.

7. Access Your Application

Once the deployment is complete, you can access your Streamlit application by navigating to the **Service URL** provided in the command-line output. If you want to use a domain name, you will need to create a CNAME with your domain provider that points to this Service URL. However at this time, Google does not recommend this for production services.

- Cloud Run domain mappings are in the preview launch stage. Due to latency issues, they are not production-ready and are not supported at General Availability. At the moment, this option is not recommended for production services.

8. Maintaining 'Always Ready' Service

To keep a Cloud Run server running without it shutting down due to inactivity, you must set a minimum number of instances to maintain a warm, ready state even when there's no traffic, or use manual scaling

to keep a specific number of instances running all the time. You should also ensure your application is configured for instance-based billing to support potential background activities and not rely on scaling to zero.

1. Configure minimum instances.

Change the `gcloud run deploy ${IMAGE_NAME}` command above, and include `--cpu-boost` \ flag when deploying

OR

```
gcloud run services update SERVICE_NAME --min-instances=1
```

OR

- Navigate to your Cloud Run service: in the Google Cloud console.
- Edit the service.
- Adjust the "Minimum number of instances" setting: to a value greater than zero. This tells Cloud Run to keep at least that many instances running, even when they are idle.
- Save the changes: to deploy the updated configuration.

2. USE CPU boost

You can use CPU boost tag to double CPU effort for the first 10 seconds. This helps speed the application startup when running a cold start.

Change the `gcloud run deploy ${IMAGE_NAME}` command above, and set the flag `--min-instances 1` \ when deploying

3. **Consider CPU always allocated** Additionally, this feature ensures that a container instance's CPU is fully available for background processing, not just during request handling. It is typically used in combination with minimum instances.

- In Google Cloud Console: In the service editor, navigate to the Container, networking, security section. In the Container tab, select CPU is always allocated.
- In gcloud CLI:

```
gcloud run services update SERVICE_NAME --cpu-throttling --cpu-throttling=never
```

3. Other important considerations

- **Instance-based billing**: Make sure your Cloud Run service is configured for instance-based billing so you can run background activities and not risk your service scaling to zero.
- **Costs**: Keeping instances running will incur costs, even when they are idle. You should weigh your performance requirements against your budget.

- **Application design**: Optimize your application by moving initialization logic to the container's global scope to reduce latency for subsequent requests after a cold start.
- **Monitor memory and CPU**: Keep an eye on your memory limits and CPU utilization to prevent application crashes or unexpected shutdowns.